NVIDIA
SDET Pre-Internship
Self-Study Program

Deep Learning Infrastructure Track — Local GPU Edition

Phases 4–7: AI Inference · TRT-LLM · Dynamo · NeMo + Integration

April 16 → May 18, 2025 | 4.5 Weeks | 4 Phases

All work runs on your local RTX 5070 Ti (16 GB VRAM, 64 GB RAM, Ryzen 9950X3D).
No Google Colab. Cursor is your primary IDE.

Beginner-friendly · Application-first · One project, six technologies

What Changed and Why

The original guide (Phases 1–3) was written around Google Colab free-tier T4s and TRT-LLM v0.17.0. Three major shifts demand a rewrite of Phases 4–7:

1. TRT-LLM's paradigm shift. The old workflow — convert checkpoint → trtllm-build → deploy engine — is now legacy. TRT-LLM v1.2 GA (current stable) introduced a PyTorch-native LLM API that loads HuggingFace models directly with zero conversion steps. The CLI tools are now `trtllm-serve`, `trtllm-bench`, and `trtllm-eval`. This guide teaches the current workflow, not the deprecated one.

2. NVIDIA Dynamo reached production 1.0 at GTC 2026 (March 16). It is no longer a "read about it" topic — it is an open-source, runnable system that works on a single GPU. It replaces Triton Inference Server as NVIDIA's distributed inference framework. Your team uses it. You should have run it before day one.

3. NeMo decomposed into 19+ repositories. The monolithic NVIDIA/NeMo repo is now speech-only. Evaluator, Curator, and Guardrails each have their own repos under github.com/NVIDIA-NeMo. Several of these run on consumer GPUs and are directly relevant to SDET work.

4. Your hardware is better than Colab. The RTX 5070 Ti with 16 GB VRAM fits TinyLlama-1.1B in FP16 (~3 GB), LLaMA-3-8B in INT4 AWQ (~7 GB), and even 14B models quantised to INT4 (~11 GB). With 64 GB system RAM, you can also offload layers to CPU for larger experiments. There is no reason to use Colab.

Your Repo — Continuing from Phases 1–3

You are continuing in the same `inference-sandbox` GitLab repo you created in Phase 1 and built up through Phases 2 and 3. Everything you have already — your benchmark.py, Dockerfile, docker-compose.yml, Kubernetes manifests, CI pipeline, Learning Log — stays. Phases 4–7 add new files alongside what exists, and extend benchmark.py with new backends. Do not start a fresh repo. The entire point of the throughline project is that it grows week by week from a single Python script into a full inference testing system. Your git history tells that story.

If you need to reorganise anything from Phases 1–3 (for example, moving test files into the `tests/` directory or renaming scripts), do it in a dedicated MR before starting Phase 4. Keep your main branch clean and your commit history readable.

Environment change: from venv to conda

Phases 1–3 may have used a plain Python virtualenv. From Phase 4 onward, use a conda environment named `nvidia` instead. Conda handles CUDA toolkit versions and binary dependencies (like PyTorch nightly builds) more reliably than pip alone, which matters once you start working with TRT-LLM and Dynamo containers. The setup instructions in Step 3 below reflect this change.

Testing: pytest everywhere

The original guide used a `--self-test` flag inside benchmark.py for quick validation. From Phase 4 onward, all testing is done through **pytest**. This means: every testable assertion lives in a file under `tests/`, you run them with `pytest` (not by passing flags to benchmark.py), and your CI pipeline calls pytest directly. If your Phase 1–3 `--self-test` logic still exists, migrate those assertions into `tests/test_benchmark_basics.py` as a cleanup MR before starting Phase 4. This is a small refactor with a large payoff — every new test you write from here on slots cleanly into the same harness.

Your Cursor Setup for These Phases

Before starting Phase 4, configure Cursor as your AI-assisted development environment. Your mentor specifically asked you to learn how to use AI tools effectively — this is that.

Cursor Rules

Create `.cursor/rules/` in your inference-sandbox repo. Add the following rule files:

.cursor/rules/nvidia-stack.mdc — Always active context:

```
---
description: NVIDIA inference stack context
alwaysApply: true
---
This project benchmarks AI inference using NVIDIA's stack.
Tech: Python 3.11, Docker, Kubernetes, TensorRT-LLM v1.2+,
      NVIDIA Dynamo 1.0, NeMo.
GPU: RTX 5070 Ti (sm_120, 16GB VRAM, Blackwell consumer).
Environment: conda env named 'nvidia'. All tests via pytest.
Constraints: CUDA 12.8+, PyTorch nightly (sm_120 requires it),
             Linux/WSL2 only for TRT-LLM.
When suggesting GPU code, always check VRAM fits within 14GB
  (leave 2GB headroom).
When suggesting TRT-LLM code, use the new LLM() API, NOT the
  legacy trtllm-build workflow.
```

.cursor/rules/sdet-mindset.mdc — Testing-first guidance:

```
---
description: SDET testing patterns
alwaysApply: true
---
Every function should be testable. Every benchmark should have assertions.
Use pytest with markers: @pytest.mark.gpu (skip if no GPU),
                         @pytest.mark.slow, @pytest.mark.smoke.
Prefer parametrized tests. Always test edge cases for VRAM limits.
When writing benchmarks, always measure: TTFT, TPOT, throughput (tokens/sec).
Log results as structured JSON for regression comparison.
```

How to Use Cursor Without Becoming Dependent on It

Your mentor's guidance is precise: "read through what it generates, understand the why, and verify the correctness." The rule is simple:

- **Use Cursor FOR:** exploring unfamiliar codebases (e.g., TRT-LLM source), generating test scaffolding you will then modify, explaining error messages from CUDA/Docker/K8s, iterating on code you already understand conceptually.
- **Do NOT use Cursor FOR:** writing code you have never written by hand before (write it yourself first, then ask Cursor to review), skipping the "why" (if you cannot explain what the code does line-by-line, you do not understand it), replacing documentation reading (Cursor hallucinates API details — always verify against official NVIDIA docs).

The Verification Habit

After every Cursor-generated code block, before running it, read it line by line and write a one-sentence comment above each section explaining what it does. If you cannot write the comment, you do not understand the code. This is not busywork — it is the fastest path to real comprehension.

Useful Cursor Workflows for Infrastructure Work

• **Codebase exploration:** Clone github.com/NVIDIA/TensorRT-LLM. Open it in Cursor. Use Chat (Ctrl+L) with @codebase to ask: “Trace how a generate() call flows from the Python LLM API to the CUDA kernel execution.” Cursor will search the repo and explain the call chain. Verify by reading the files it references.

• **Error diagnosis:** When a Docker build or TRT-LLM command fails, paste the full error into Chat with the relevant file attached (@file). Ask “What went wrong and what is the fix?” Then verify the fix against the official docs before applying.

• **Test generation:** After writing a function, use Agent Mode (Cmd+I) with the prompt: “Write pytest tests for this function. Include edge cases for empty input, VRAM overflow, and dtype mismatch. Use parametrize.” Review every assertion before committing.

Your Local GPU Environment Setup

Do this setup ONCE before Phase 4. Everything downstream depends on it.

Step 1: Verify Your NVIDIA Driver

```
nvidia-smi
# You need: Driver 570+ (575+ preferred for Blackwell)
# Should show: NVIDIA GeForce RTX 5070 Ti, 16384 MiB memory
# CUDA Version shown here is the MAX supported, not installed
```

If your driver is older than 570, update via NVIDIA's official .run installer or your distro's package manager. The open-kernel driver (`nvidia-open`) is preferred for RTX 50-series on Linux.

Step 2: Install CUDA Toolkit 12.8+ (13.0 recommended)

```
# Check current CUDA:
nvcc --version
# If missing or < 12.8, install CUDA 13.0 from NVIDIA:
#   https://developer.nvidia.com/cuda-downloads
#   Select: Linux > x86_64 > Ubuntu > 24.04 > deb (network)
# After install, verify:
nvcc --version   # Should show 12.8+ or 13.0
```

Step 3: Install PyTorch Nightly (Required for sm_120)

The RTX 5070 Ti uses compute capability sm_120 (Blackwell). Stable PyTorch does NOT support this yet — you need nightly builds.

```
# Create a conda environment for inference work:
conda create -n nvidia python=3.11 -y
conda activate nvidia

# Install PyTorch nightly with CUDA 12.8 or 13.0:
pip install --pre torch torchvision torchaudio \
    --index-url https://download.pytorch.org/whl/nightly/cu128

# Verify GPU access:
python -c "import torch; print(torch.cuda.is_available()); \
           print(torch.cuda.get_device_name())"
# Should print: True
#               NVIDIA GeForce RTX 5070 Ti
```

> *If this fails with a "no kernel image" error, you need a newer nightly. Check https://download.pytorch.org/whl/nightly/cu128/ or try the cu130 variant.*

Step 4: Install the NVIDIA Container Toolkit (for Docker GPU access)

```
# This lets Docker containers access your GPU
# Follow: https://docs.nvidia.com/datacenter/cloud-native/
#         container-toolkit/latest/install-guide.html
# After install, verify:
docker run --rm --gpus all \
    nvidia/cuda:12.8.0-base-ubuntu22.04 nvidia-smi
# Should show your RTX 5070 Ti inside the container
```

Step 5: Install Ollama (Your First Local Inference Tool)

```
curl -fsSL https://ollama.ai/install.sh | sh

# Verify GPU offload:
ollama run qwen3:0.6b "Hello, what GPU am I running on?"
# Check ollama logs or run `ollama ps` to confirm GPU layers loaded

# Enable Flash Attention for better performance:
echo 'OLLAMA_FLASH_ATTENTION=1' | sudo tee -a /etc/environment
```

■ Write all setup steps you ran into a file docs/local_gpu_setup.md in your repo. Include exact commands, versions, and any errors you hit. This is your reproducibility artifact AND your troubleshooting reference for the next 5 weeks.

✓ Setup is done when: nvidia-smi shows your GPU, torch.cuda.is_available() returns True, docker run --gpus all works, and ollama run generates text using GPU.

PHASE 4: AI INFERENCE CONCEPTS

April 16 – April 30 (2 weeks) | Hands-on inference on YOUR GPU — no cloud, no notebooks

---

Why This Phase Exists

Your ML class teaches how models train. Your job at NVIDIA is about how they run AFTER training — inference. This phase connects those worlds. But unlike the original guide that ran toy experiments on a borrowed T4, you have a real GPU sitting under your desk. You will load real models, measure real latency, observe real VRAM pressure, and write real tests — all locally, all in Cursor, all committed to your inference-sandbox repo.

Core Concepts

The Core Concepts from the original guide (Two Phases of LLM Inference, Latency vs Throughput, Quantisation, KV Cache) remain accurate and essential. Re-read them. The concepts have not changed — only the tools have. Below is a consolidated reference combining the original explanations with updates for the current landscape.

The Two Phases of LLM Inference

```
# PREFILL PHASE
# Input:  your full prompt ("What is the capital of France?")
# Process: entire prompt processed in ONE forward pass
# Output:  KV Cache populated; first output token generated
# Metric:  TTFT (Time To First Token)
# Characteristic: COMPUTE-BOUND -- bottlenecked by raw GPU FLOPS
#
# DECODE PHASE
# Process: generates ONE token at a time, each requiring a forward pass
#          Each step reads the KV cache from all previous tokens
# Output:  a stream of tokens until EOS or max_tokens
# Metric:  TPOT (Time Per Output Token)
# Characteristic: MEMORY-BANDWIDTH-BOUND -- bottlenecked by VRAM read speed
#
# Why this split matters for your job:
#   Optimising TTFT = make prefill faster (Flash Attention, chunked prefill)
#   Optimising TPOT = make decode faster (paged KV cache, speculative decoding)
#   These require DIFFERENT techniques --
#   Dynamo disaggregates them for this exact reason
```

Latency vs Throughput — Different Goals

```
# LATENCY = how long a single user waits for their response
#   Measured as: TTFT, TPOT, total response time
#   Minimised by: small batch sizes, fast hardware
#
# THROUGHPUT = tokens generated per second across ALL users
#   Maximised by: large batch sizes, continuous batching
#
# The tension:
#   Batch size  1: latency=10ms/token, throughput=100 tokens/sec
#   Batch size 32: latency=40ms/token, throughput=800 tokens/sec
#
# Serving systems set a target latency SLO and maximise throughput
# within it. Your job as an SDET is to verify both metrics stay
# within agreed bounds.
```

Quantisation — Making Models Faster and Smaller

Neural network weights are floating-point numbers. Using fewer bits per number reduces memory usage and speeds up computation — at some cost to accuracy.

Format	Bits	Bytes/Param	Rel. Speed	Accuracy Loss	NVIDIA Support
FP32	32	4.0	1x (baseline)	None (baseline)	All GPUs
FP16	16	2.0	~2x	Negligible	All GPUs (Ampere+)
BF16	16	2.0	~2x	Negligible	Ampere+ (preferred)
INT8 (W8A8)	8	1.0	~3–4x	Small — must test	Turing+
FP8 (E4M3)	8	1.0	~4x	Very small on H100	H100+ only
INT4 (GPTQ)	4	0.5	~6x	Noticeable — test!	Ampere+

The SDET angle on quantisation: your job is not to implement quantisation — it is to test that the accuracy loss is within an acceptable bound. The key metric is perplexity. FP16 perplexity should be within ~0.5 of FP32. If it is not, the quantisation is broken or misconfigured.

KV Cache — The Memory Bottleneck

```
# KV cache size estimation (rough):
#
# kv_cache_bytes =
#     num_layers
#     x 2 (key + value)
#     x num_kv_heads
#     x head_dim
#     x seq_len
#     x bytes_per_element
#     x batch_size
#
# Example: LLaMA-3-8B, seq_len=4096, batch=32, FP16
#   = 32 layers x 2 x 8 heads x 128 dim x 4096 tokens x 32 batch x 2 bytes
#   = ~8.6 GB -- just for the KV cache, before weights (16GB for FP16)
```

The Local Inference Tool Chain

Understanding inference means understanding the tools that run it. These are ordered from simplest to most NVIDIA-specific. You will use all four in this phase and the next:

```
# Tool chain progression (you will use all four):
#
# Ollama   -> Simplest. Wraps llama.cpp. Good for "does this model work?"
#    |        One command to run any model. Abstracts everything.
# HuggingFace -> Python API. Full control over generation parameters.
#    |        Where you learn tokenisation, sampling, dtype selection.
# vLLM     -> Production serving. PagedAttention, continuous batching.
#    |        OpenAI-compatible API. Where you learn serving concepts.
# TRT-LLM  -> NVIDIA's optimised runtime. Compiled kernels, max throughput.
#             What your team builds and tests. Phase 5 focus.
#
# Each tool teaches something the previous one hides:
#   Ollama hides quantisation  -> HF makes you choose dtype
#   HF hides batching          -> vLLM teaches continuous batching
#   vLLM hides kernel fusion   -> TRT-LLM teaches compiled inference
```

VRAM Budgeting — A Skill, Not a Concept

With 16 GB, you must think about VRAM the way a systems programmer thinks about memory. Every model load is a budgeting decision:

```
# VRAM budget for RTX 5070 Ti:
#
# Total:          16,384 MB
# OS/driver:        ~500 MB (always reserved)
# PyTorch/CUDA:     ~500 MB (framework overhead)
# Available:     ~15,000 MB
# Safe target:   ~14,000 MB (leave headroom for KV cache growth)
#
# Model weight sizes (approximate):
#   1B params x FP16  =  ~2 GB
#   8B params x FP16  = ~16 GB  <- does NOT fit with overhead
#   8B params x INT4  =  ~4 GB  <- fits comfortably
#   8B params x INT8  =  ~8 GB  <- fits with room for KV cache
#  14B params x INT4  =  ~7 GB  <- fits, moderate context window
#
# KV cache grows with:
#   sequence_length x batch_size x num_layers x hidden_dim
# On long sequences, KV cache can exceed model weight size
```

Phase 4 Projects

PROJECT 4.1: BENCHMARK WITH OLLAMA — YOUR FIRST LOCAL INFERENCE TEST

Objective: Measure real inference metrics on your own GPU using the simplest possible tool.

1. Pull three models of increasing size: `ollama pull qwen3:0.6b` (~500 MB, trivially small), `ollama pull qwen3:4b` (~2.5 GB), `ollama pull qwen3:8b` (~5 GB, Q4 quantised).

2. Write a script `benchmarks/ollama_bench.py` that uses the Ollama Python client (`pip install ollama`) to: send the same 5 prompts to each model, measure TTFT (time from request to first token via streaming), measure total generation time and compute tokens/sec, record VRAM usage via `nvidia-smi --query-gpu=memory.used --format=csv` (subprocess call).

3. Output results as a formatted table AND save to a JSON Lines file.

4. Write a `tests/test_ollama_bench.py` with pytest assertions: assert TTFT > 0, assert tokens/sec > 0, assert VRAM increased after model load.

5. Branch: `feature/ollama-benchmark`. MR with description explaining your TTFT and throughput observations across the three model sizes.

✓ Done when: Script runs all three models. Results table shows clear trends: larger models have higher TTFT and lower tokens/sec. VRAM numbers match expected model sizes within ~20%.

■ *If TTFT does not increase with model size, check whether Ollama is keeping the previous model loaded. Use ollama ps to verify which model is active. Ollama unloads after 5 minutes by default.*

PROJECT 4.2: HUGGINGFACE LOCAL INFERENCE — DTYPE AND TOKENISATION CONTROL

Objective: Move from Ollama's abstraction to direct HuggingFace control — where you choose every parameter.

1. Add `transformers`, `accelerate`, and `torch` to requirements.txt.

2. Write `benchmarks/hf_bench.py` that loads TinyLlama-1.1B from HuggingFace:

```
from transformers import AutoModelForCausalLM, AutoTokenizer
import torch

model_name = "TinyLlama/TinyLlama-1.1B-Chat-v1.0"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype=torch.float16,   # You choose the dtype
    device_map="cuda"
)
```

3. Implement `measure_ttft(model, tokenizer, prompt)` using `model.generate()` with a TextIteratorStreamer — capture the timestamp of the first yielded token.

4. Implement `measure_throughput(model, tokenizer, prompts, max_new_tokens=100)` — total output tokens divided by total wall time.

5. Run benchmarks at three dtypes: `torch.float32`, `torch.float16`, `torch.bfloat16`. Record VRAM for each. Note: FP32 TinyLlama is ~4.4 GB; it fits on your GPU but watch the difference.

6. Add a `--compare-dtypes` flag that runs all three and prints a comparison table.

7. Integrate with your existing `save_results()` from Phase 1 — save results to Redis.

✓ Done when: Comparison table shows FP32 uses ~2x the VRAM of FP16/BF16. TTFT and throughput numbers are physically plausible. Results persist in Redis.

■ *BF16 vs FP16: On Ampere+ GPUs (including your Blackwell 5070 Ti), BF16 has the same exponent range as FP32, making it more numerically stable than FP16 for training. For inference, the difference is usually negligible — but your job as an SDET is to TEST that claim, not assume it. That is Project 4.3.*

PROJECT 4.3: QUANTISATION ACCURACY TEST — THE CORE SDET CONTRIBUTION

Objective: Write a quality assertion that validates model accuracy under quantisation — exactly what SDET teams do at NVIDIA.

1. Write `tests/test_quantisation.py` using pytest.

2. Load TinyLlama in three precisions: FP32, FP16, BF16.

3. Compute perplexity on a 200-sentence subset of WikiText-2:

```
from datasets import load_dataset
ds = load_dataset("wikitext", "wikitext-2-raw-v1", split="test")
# Filter to non-empty lines, take first 200
```

4. Perplexity computation: for each sentence, compute the negative log-likelihood of the model's predictions, then exponentiate the average.

5. Write assertions:

```
assert abs(ppl_fp16 - ppl_fp32) < 0.5, \
    f"FP16 drift too large: {ppl_fp16:.2f} vs FP32 {ppl_fp32:.2f}"
assert abs(ppl_bf16 - ppl_fp32) < 0.5, \
    f"BF16 drift too large: {ppl_bf16:.2f} vs FP32 {ppl_fp32:.2f}"
```

6. Add `@pytest.mark.gpu` marker. Add skip logic: `pytest.mark.skipif(not torch.cuda.is_available(), reason="No GPU")`.

7. Add a `@pytest.mark.slow` marker (perplexity computation is not fast).

8. In your Learning Log, explain: what does a perplexity of 30 mean in plain language? Why is 0.5 an appropriate tolerance? When would you tighten or loosen it?

✓ Done when: `pytest tests/test_quantisation.py -m gpu` passes. You can explain perplexity without looking it up.

■ *Perplexity is the exponential of average negative log-likelihood. A perplexity of 30 means the model is "choosing between 30 equally likely next tokens" on average. Lower = better. FP16 should be within ~0.5 of FP32 for a well-behaved cast — if it is not, something is broken. This threshold IS the quality gate. Setting it is an engineering judgment call — and that judgment is the SDET's job.*

PROJECT 4.4: VRAM PRESSURE AND KV CACHE OBSERVATION

Objective: Directly observe the memory behaviour that motivates paged KV caches in TRT-LLM and Dynamo.

1. Write `benchmarks/vram_observer.py`.

2. Load TinyLlama-1.1B in FP16.

3. Record `torch.cuda.memory_allocated()` at each checkpoint: after model load (before any generation), after generating 128, 256, 512, and 1024 tokens.

4. Plot VRAM vs sequence length using matplotlib. Save as `docs/vram_vs_seqlen.png`.

5. Repeat with batch sizes [1, 2, 4, 8] at sequence length 512. Plot VRAM vs batch size. Save as `docs/vram_vs_batch.png`.

6. Write a section in LEARNING_LOG.md answering:

• Why does VRAM grow with sequence length? (KV cache accumulation)

• Why does VRAM grow with batch size? (each request gets its own KV cache)

• At what sequence length would your 16 GB GPU run out for TinyLlama? Estimate from your plot.

• What would “paged” KV cache do differently? (allocate in blocks, like virtual memory pages, instead of contiguous pre-allocation)

✓ **Done when: Both plots exist in repo. Learning Log answers are accurate and in your own words.**

■ *If VRAM does not grow linearly with sequence length, you may be measuring at the wrong point. The KV cache grows during the DECODE phase (token by token), not during prefill. Measure after model.generate() completes, not before.*

PROJECT 4.5: vLLM — PRODUCTION SERVING ON YOUR LOCAL GPU

Objective: Run a production-grade inference server locally and understand the concepts that separate “running a model” from “serving a model.”

1. Install vLLM: `pip install vllm`

2. Start a vLLM server for TinyLlama:

```
python -m vllm.entrypoints.openai.api_server \
    --model TinyLlama/TinyLlama-1.1B-Chat-v1.0 \
    --dtype float16 \
    --max-model-len 2048 \
    --port 8000
```

3. Write `benchmarks/vllm_bench.py` that uses the OpenAI Python client to hit your local server:

```
from openai import OpenAI
client = OpenAI(base_url="http://localhost:8000/v1", api_key="dummy")
```

4. Send 10 concurrent requests (use asyncio + aiohttp or concurrent.futures.ThreadPoolExecutor). Measure: individual TTFT per request, system-wide throughput (total tokens / total wall time), compare to HuggingFace sequential throughput from Project 4.2.

5. Observe the latency-throughput tradeoff: send [1, 4, 8, 16] concurrent requests. Plot TTFT vs concurrency AND throughput vs concurrency.

6. Add a `--vllm` flag to your main benchmark.py that runs benchmarks against a vLLM endpoint.

7. Write in your Learning Log: What is continuous batching? What is PagedAttention? Why does vLLM achieve higher throughput than naive HuggingFace generate()?

✓ Done when: vLLM server runs on your GPU. Concurrent benchmark shows throughput increasing with concurrency (up to a point). TTFT increases with concurrency. You can explain why both happen.

■ *vLLM's PagedAttention manages KV cache in fixed-size blocks (like OS virtual memory pages) instead of allocating one contiguous chunk per sequence. This eliminates fragmentation and allows serving more concurrent requests in the same VRAM — exactly the problem you observed in Project 4.4.*

PROJECT 4.6: INTEGRATE BENCHMARKS INTO benchmark.py

Objective: Unify all Phase 4 work into your throughline project.

1. Refactor benchmark.py to support multiple backends via a `--backend` flag:

```
python benchmark.py --backend ollama --model qwen3:0.6b
python benchmark.py --backend huggingface --model TinyLlama/TinyLlama-1.1B-Chat-v1.0
python benchmark.py --backend vllm --endpoint http://localhost:8000/v1
```

2. All backends should produce the same output schema: `{"backend": str, "model": str, "ttft_ms": float, "throughput_tps": float, "vram_mb": int, "timestamp": str}`.
3. Update `save_results()` to store backend-specific results. Update `--dump-results` to filter by backend.
4. Add a `pytest -m smoke` marker for quick validation of whatever backends are available (skip gracefully if vLLM server is not running, if no GPU, etc.).
5. Rebuild your Docker image. Verify CI passes.

✓ Done when: benchmark.py is a unified benchmarking tool with three backends. Docker image builds. CI passes without a GPU (all GPU tests skip gracefully).

■ *This unified interface is not just code cleanliness — it is the pattern used by real benchmark harnesses at NVIDIA. The benchmark tool is backend-agnostic; the backend is swappable. In Phase 5, you will add TRT-LLM as a fourth backend, and in Phase 6, a Dynamo endpoint. Same interface, different engine.*

Phase 4 Checkpoint

■ Before Moving to Phase 5 you should be able to:

1. Explain TTFT vs TPOT without notes.
2. Describe the latency-throughput tradeoff at different concurrency levels using your own data.
3. Explain what quantisation trades off and how perplexity measures it.
4. Explain why KV cache is a memory bottleneck using your VRAM plots.
5. Explain what PagedAttention does and why it matters.
6. Run `benchmark.py --backend huggingface` on your GPU and get plausible numbers.

PHASE 5: TRT-LLM HANDS-ON

April 30 – May 10 (11 days) | Build, serve, and test with NVIDIA's production inference runtime — on your own GPU

What TRT-LLM Is and Why It Exists

PyTorch is a research framework — flexible but not optimised for production inference. TensorRT-LLM is NVIDIA's production-grade alternative: it takes a trained model and runs it with custom CUDA kernels for attention, KV cache management, batching, and speculative decoding. A TRT-LLM engine for LLaMA-3-8B is typically 3–5x faster than the equivalent PyTorch model.

What Changed Since the Original Guide

> **■■ Critical: The original guide taught TRT-LLM v0.17.0's workflow: convert checkpoint → trtllm-build → deploy engine. That entire workflow is now legacy. TRT-LLM v1.0 (July 2025) introduced a PyTorch-native backend, and v1.2 GA (current stable) made it the default.**

```
# OLD (legacy, pre-v1.0) -- DO NOT USE as your primary workflow:
# python convert_checkpoint.py -> trtllm-build -> run.py
#
# NEW (v1.2+ current) -- this is what you learn:
from tensorrt_llm import LLM, SamplingParams

llm = LLM(model="TinyLlama/TinyLlama-1.1B-Chat-v1.0")
output = llm.generate(
    ["Explain inference in one sentence."],
    SamplingParams(temperature=0.8, max_tokens=50)
)
print(output[0].outputs[0].text)
```

The key CLI tools are:

- **trtllm-serve** — launches an OpenAI-compatible API server (replaces manual Triton setup)
- **trtllm-bench** — built-in benchmarking tool (measures TTFT, TPOT, throughput)
- **trtllm-eval** — runs accuracy evaluation against standard benchmarks
- **AutoDeploy** — beta feature that auto-optimises any HuggingFace model without manual config

TRT-LLM Installation on Your RTX 5070 Ti

Docker is the most reliable path for Blackwell consumer GPUs. The NGC container has all dependencies pre-configured.

```
# Option A: NGC Docker container (RECOMMENDED)
docker pull nvcr.io/nvidia/tensorrt-llm/release:1.2.0

# Run with GPU access:
docker run --rm -it --gpus all \
    -v $(pwd):/workspace \
    nvcr.io/nvidia/tensorrt-llm/release:1.2.0 bash

# Option B: pip install (works but more fragile on sm_120)
# Requires: CUDA 12.8+, PyTorch nightly, Python 3.11
pip install tensorrt_llm --extra-index-url https://pypi.nvidia.com
```

■■ Check your TRT-LLM version against the latest release notes at nvidia.github.io/TensorRT-LLM/release-notes.html before starting. If v1.3.0 has gone GA by the time you read this, use that instead. Blackwell support improves with every release.

■ If Docker is your path (recommended), add a new Dockerfile to your repo: `Dockerfile.trtllm` that extends the NGC base image with your project files. This is realistic — NVIDIA teams build custom images on top of NGC bases.

Phase 5 Projects

PROJECT 5.1: YOUR FIRST TRT-LLM INFERENCE — THE NEW WAY

Objective: Go from zero to running TRT-LLM inference on your GPU using the current (v1.2+) API.

1. Enter your TRT-LLM environment (Docker container or pip install).
2. Write `benchmarks/trtllm_basic.py`:

```
from tensorrt_llm import LLM, SamplingParams

# Load TinyLlama -- TRT-LLM handles optimisation automatically
llm = LLM(model="TinyLlama/TinyLlama-1.1B-Chat-v1.0")

prompts = [
    "Explain neural network quantisation in simple terms.",
    "What is the difference between latency and throughput?",
    "Write a Python function that reverses a string.",
]

outputs = llm.generate(prompts, SamplingParams(
    temperature=0.0,   # Greedy -- deterministic
    max_tokens=100
))

for prompt, output in zip(prompts, outputs):
    print(f"Prompt: {prompt}")
    print(f"Output: {output.outputs[0].text}\n")
```

3. Run it. Record the output and the load time.
4. Time the first call (includes model compilation/optimisation) vs subsequent calls.
5. Try with temperature=0.8 and top_p=0.9 to see non-deterministic output.
6. Save ALL commands and output to `docs/trtllm_first_run.md`.

✓ Done when: TRT-LLM generates coherent text for all 3 prompts on your GPU. You have documented exact commands, versions, and timing.

■ *The first generate() call will be slow — TRT-LLM is compiling and optimising the model graph. Subsequent calls will be much faster. This is analogous to JIT compilation. If it fails with a CUDA error, check that your TRT-LLM version supports sm_120.*

PROJECT 5.2: trtllm-serve — RUNNING A PRODUCTION API SERVER

Objective: Serve a model via TRT-LLM's built-in OpenAI-compatible API server.

1. Start the server:

```
trtllm-serve TinyLlama/TinyLlama-1.1B-Chat-v1.0 \
    --host 0.0.0.0 --port 8001
```

2. In another terminal, hit it with curl:

```
curl http://localhost:8001/v1/completions \
    -H "Content-Type: application/json" \
    -d '{"model": "TinyLlama/TinyLlama-1.1B-Chat-v1.0",
         "prompt": "The capital of France is",
         "max_tokens": 20, "temperature": 0}'
```

3. Update benchmark.py with a `--backend trtllm-serve` option that hits this endpoint using the OpenAI client (identical interface to your vLLM backend — that is the point of OpenAI-compatible APIs).

4. Run your concurrent benchmark from Project 4.5 against trtllm-serve. Compare throughput to vLLM.

5. Write in your Learning Log: how does the API match/differ from vLLM's API? What metrics does the server expose?

✓ Done when: trtllm-serve runs on your GPU. `benchmark.py --backend trtllm-serve` **produces throughput numbers. You have a direct comparison to vLLM for the same model.**

■ *The OpenAI-compatible API is now the standard interface for inference servers. vLLM, TRT-LLM, and Dynamo all expose it. Your benchmark tool's backend abstraction pays off here — same client code, different server.*

PROJECT 5.3: trtllm-bench — NVIDIA'S OWN BENCHMARKING TOOL

Objective: Learn and run the official NVIDIA benchmarking tool, then compare its results to yours.

1. Run trtllm-bench on TinyLlama:

```
trtllm-bench --model TinyLlama/TinyLlama-1.1B-Chat-v1.0 \
    --dataset_type synthetic \
    --num_requests 100 \
    --concurrency 1 4 8
```

2. Study the output: it reports TTFT (p50, p90, p99), TPOT, throughput, and request latencies.

3. Compare these numbers to your benchmark.py output for the same model and concurrency levels.

4. Write `tests/test_bench_sanity.py`: run your benchmark and trtllm-bench, assert that your throughput numbers are within 25% of the official tool's. If they are not, your measurement methodology has a bug.

5. In your Learning Log, explain: what is p50 vs p90 vs p99 latency? Why do production SLOs typically use p99 rather than average?

✓ Done when: trtllm-bench runs successfully. Your custom benchmark numbers are within 25% of the official tool. You understand percentile latencies.

■ *p99 latency means 99% of requests complete within this time. It captures worst-case user experience. Average latency hides tail latency spikes. Production SLOs at NVIDIA are defined in p99 — your performance tests should assert on p99, not average. If your benchmark only reports averages, it is not production-quality.*

PROJECT 5.4: CORRECTNESS REGRESSION TEST — TRT-LLM vs HUGGINGFACE

Objective: Write the test that validates TRT-LLM produces outputs matching the HuggingFace reference — the core SDET deliverable.

1. Write `tests/test_trtllm_correctness.py`.

2. Define a list of 10 prompts covering: factual questions, code generation, reasoning, short and long expected outputs.

3. For EACH prompt, with greedy decoding (temperature=0):

• Run with HuggingFace: `model.generate(..., do_sample=False)`

• Run with TRT-LLM: `llm.generate(..., SamplingParams(temperature=0))`

• Compare output token IDs (not just text — tokenisation differences can hide mismatches)

4. Assert at least 9/10 prompts produce identical output tokens. Log ALL mismatches with: full prompt, expected output (HF), actual output (TRT-LLM), token-level diff (first divergence point).

5. Add `@pytest.mark.gpu` marker and skip logic.

6. Write in your Learning Log: why would greedy decoding produce different outputs between two implementations of the same model? (Hint: numerical precision, tokeniser config, special token handling, KV cache implementation differences.)

✓ Done when: Test passes at ≥9/10. Mismatches are logged with enough detail to diagnose. You can explain potential mismatch sources.

■ *Even with greedy decoding and identical settings, floating-point non-determinism means the probability distributions are slightly different. At "close call" positions where two tokens have nearly identical probabilities, different implementations can diverge. This is expected — your test tolerates it (9/10, not 10/10). Debugging the 1/10 mismatch is where SDET expertise lives.*

PROJECT 5.5: PERFORMANCE REGRESSION TEST WITH GOLDEN BASELINE

Objective: Build the automated performance gate that catches regressions.

1. Run your benchmark against trtllm-serve at batch sizes [1, 2, 4]:

```
python benchmark.py --backend trtllm-serve --concurrency 1
python benchmark.py --backend trtllm-serve --concurrency 4
```

2. Save these measurements as `golden_baseline.json`:

```
{
    "model": "TinyLlama-1.1B",
    "gpu": "RTX_5070_Ti",
    "trtllm_version": "1.2.0",
    "measurements": {
        "batch_1": {"ttft_p99_ms": 45.2, "throughput_tps": 120.7},
        "batch_4": {"ttft_p99_ms": 78.3, "throughput_tps": 340.5}
    }
}
```

✓ Done when: Regression test correctly fails when performance degrades. golden_baseline.json is committed to repo with GPU and TRT-LLM version metadata.

■ *The 15% tolerance exists because GPU performance has natural variance — thermal throttling, background processes, memory contention. Too tight a tolerance causes flaky tests. Too loose and you miss real regressions. 10–20% is the standard range for ML performance gates at NVIDIA. Document your choice of tolerance and why.*

PROJECT 5.6: EXPLORE THE LEGACY BUILD PIPELINE (OPTIONAL BUT VALUABLE)

Objective: Understand the old workflow so you can work with existing NVIDIA codebases that use it. Many production systems at NVIDIA were built with the legacy pipeline.

1. Clone the TRT-LLM examples:

```
git clone https://github.com/NVIDIA/TensorRT-LLM
cd TensorRT-LLM/examples/llama
```

2. Use Cursor to explore the codebase: “How does convert_checkpoint.py transform HuggingFace weights into TRT-LLM format? What does trtllm-build do to those weights?”

3. Run the legacy build for TinyLlama (if your environment supports it):

```
python convert_checkpoint.py \
    --model_dir TinyLlama/TinyLlama-1.1B-Chat-v1.0 \
    --output_dir /tmp/trtllm_ckpt --dtype float16

trtllm-build --checkpoint_dir /tmp/trtllm_ckpt \
    --output_dir /tmp/trtllm_engine \
    --max_batch_size 4 --max_input_len 512 --max_seq_len 1024
```

4. Compare the engine output to the LLM API output. Are they identical for greedy decoding?

5. Measure build time. Measure the engine file sizes. Write observations in your Learning Log.

6. In your Learning Log, explain: when would the legacy pipeline be preferred over the new LLM API? (Answer: when you need precise control over engine configuration, when integrating with custom CUDA kernels, when deploying to specific hardware profiles.)

✓ Done when: You understand both the new and legacy workflows. You can explain when each is appropriate. Learning Log documents the differences.

Phase 5 Checkpoint

■ Before Moving to Phase 6 you should be able to:

1. Use the TRT-LLM LLM API to load and generate from a model.

2. Serve a model with trtllm-serve and hit it with OpenAI-compatible requests.

3. Explain what trtllm-bench measures and what p99 latency means.

4. Have a passing correctness test that compares TRT-LLM to HuggingFace.

5. Have a performance regression test with a golden baseline.

6. Explain the difference between the new LLM API and the legacy engine build pipeline.

PHASE 6: NVIDIA DYNAMO

May 10 – May 16 (6 days) | Run and understand the distributed inference framework your team deploys

---

What Dynamo Is and Why It Matters Now

Dynamo is NVIDIA's open-source distributed inference serving framework. It reached production 1.0 at GTC 2026 (March 16, 2026). It is the successor to Triton Inference Server — evolving from single-node multi-framework serving to distributed multi-node orchestration for LLMs. Built in Rust for performance with Python for extensibility.

Dynamo is not just "another serving framework." It introduces architectural concepts that fundamentally change how inference services are designed:

```
# Traditional serving (vLLM, TRT-LLM standalone):
#
# [User] -> [Single GPU running both prefill AND decode]
#
# Problem: prefill is COMPUTE-bound, decode is MEMORY-BANDWIDTH-bound.
# Same GPU cannot be optimised for both simultaneously.
# Result: wasted resources, suboptimal latency OR throughput.
#
#
# Dynamo's disaggregated serving:
#
# [User] -> [Router] -> [Prefill GPU pool] -> KV transfer -> [Decode GPU pool]
#                       (compute-optimised)    (NIXL)         (bandwidth-optimised)
#
# Each pool scales independently based on traffic patterns.
# The router knows which decode GPU already has relevant KV cache.
# Result: up to 2x+ throughput on Llama 70B vs co-located serving.
```

Running Dynamo Locally

Yes, Dynamo runs on a single GPU. The single-GPU mode does not demonstrate disaggregation (you need 2+ GPUs for that), but it does demonstrate the serving APIs, the router, and the deployment patterns your team uses.

```
# Quick start (from NVIDIA docs):
docker pull nvcr.io/nvidia/ai-dynamo/dynamo:1.0.1

docker run --rm -it --gpus all \
    -p 8080:8080 \
    nvcr.io/nvidia/ai-dynamo/dynamo:1.0.1 bash

# Inside the container, Dynamo examples are pre-installed
# The quickstart uses Qwen3-0.6B (~2GB VRAM, trivial on your 5070 Ti)
```

Phase 6 Projects

PROJECT 6.1: RUN DYNAMO'S QUICKSTART

Objective: Get a Dynamo inference endpoint running on your GPU and understand its architecture.

1. Pull the Dynamo container and enter it.

2. Follow the official quickstart at docs.nvidia.com/dynamo/getting-started/: start an etcd instance (Dynamo's service registry), start the Dynamo HTTP frontend, start a worker with a model (Qwen3-0.6B or TinyLlama).

3. Hit the endpoint with curl and the OpenAI client.

4. Compare the API to trtllm-serve's API. What is the same? What is different?

5. Use `dynamo list` or equivalent CLI to see running components.

6. Document the full startup process in `docs/dynamo_quickstart.md`.

✓ Done when: Dynamo serves inference requests on your GPU. You can hit the endpoint with curl and the OpenAI client. Startup process is documented.

■ *Dynamo's architecture is more complex than a standalone server — it has a frontend, router, and workers that communicate via a service mesh. Understanding these components is key to understanding the failure modes you will test in your internship.*

PROJECT 6.2: BENCHMARK DYNAMO vs TRT-LLM STANDALONE

Objective: Use your unified benchmark tool to compare Dynamo to bare TRT-LLM.

1. Add `--backend dynamo` to your benchmark.py:

```
python benchmark.py --backend dynamo --endpoint http://localhost:8080/v1
```

2. Run identical benchmarks against both trtllm-serve and Dynamo for the same model.

3. On a single GPU, Dynamo should perform similarly to TRT-LLM standalone (Dynamo's advantages emerge at scale). Verify this — if there is a significant gap, investigate why.

4. Record results in Redis. Generate a comparison report.

5. Write in your Learning Log: where would Dynamo's disaggregated prefill/decode architecture outperform standalone TRT-LLM? Why does single-GPU mode not show this advantage?

✓ Done when: benchmark.py supports Dynamo as a backend. Comparison data exists. You understand why single-GPU results do not show disaggregation benefits.

PROJECT 6.3: EXPLORE DYNAMO SOURCE CODE IN CURSOR

Objective: Use Cursor to understand how Dynamo orchestrates inference at a code level.

1. Clone github.com/ai-dynamo/dynamo and open in Cursor.

2. Use Chat with @codebase to answer:

• "How does the router decide which worker receives a request?"

• "Where is the disaggregated prefill/decode logic implemented?"

• “How does NIXL transfer KV cache between GPUs?”

• “What happens when a worker crashes mid-generation?”

3. For each answer Cursor gives, find the actual source file and verify.

4. Write `docs/dynamo_architecture.md`:

• Section 1: ASCII diagram of Dynamo’s component architecture

• Section 2: Request lifecycle (HTTP request → token output, every hop)

• Section 3: How disaggregated prefill/decode works (in your own words)

• Section 4: What NIXL does and why it uses RDMA/NVLink

• Section 5: 5 specific questions for your mentor about Dynamo internals

✓ Done when: Architecture document is accurate. You can trace a request through the system. Your questions show genuine understanding of the system’s design.

■ *The open questions section matters as much as the diagrams. Walking into your first 1:1 with 5 specific, well-formed questions about Dynamo signals exactly the right level of preparation. “How does NIXL handle KV transfer when the decode node has less VRAM than the prefill node sent?” is a better question than “What is NIXL?”*

PROJECT 6.4: WRITE A DYNAMO HEALTH CHECK TEST

Objective: Write the kind of infrastructure test you will write in your internship.

1. Write `tests/test_dynamo_health.py`:

• Start the Dynamo endpoint (or assume it is running)

• Assert the API responds to /v1/models with the expected model

• Assert a simple generation request completes within a timeout

• Assert the response schema matches the OpenAI spec

• If the endpoint is not running, skip with a clear message

2. Add a `@pytest.mark.integration` marker (these tests require a running service).

3. This is a health check — the simplest infrastructure test. Your internship will build on this pattern to test failure recovery, scaling behaviour, and distributed correctness.

✓ Done when: Health check passes when Dynamo is running. Skips cleanly when it is not. Test output clearly states what passed and what was checked.

Phase 6 Checkpoint

■ Before Moving to Phase 7 you should be able to:

1. Start a Dynamo inference endpoint locally.

2. Explain disaggregated prefill/decode and why it exists.

3. Explain what NIXL does.

4. Use your benchmark tool to compare Dynamo to TRT-LLM standalone.

5. Trace a request through Dynamo’s architecture.

6. Have specific, well-formed questions for your mentor about Dynamo internals.

PHASE 7: NeMo ECOSYSTEM + FINAL INTEGRATION

May 16 – May 18 (3 days) | Understand the training side of the stack and tie everything together

---

Why NeMo Matters for an SDET

NeMo produces the model checkpoints that TRT-LLM compiles and Dynamo serves. The models you test had to be trained, fine-tuned, and evaluated before reaching you. Understanding where models come from means you can ask better questions about model quality issues, and NeMo's evaluation and data curation tools are directly relevant to test engineering.

The NeMo Ecosystem (2025–2026)

NeMo is no longer one repository. It has been decomposed into 19+ specialised repos under github.com/NVIDIA-NeMo. The ones relevant to your SDET role:

```
# NeMo ecosystem (as of April 2025):
#
# NVIDIA-NeMo/NeMo        -> Speech AI only (ASR, TTS). NOT general LLM training.
# NVIDIA-NeMo/Evaluator   -> Model evaluation (MMLU, GSM8K, etc.) -- YOUR testing tool
# NVIDIA-NeMo/Curator     -> Training data curation -- data quality = model quality
# NVIDIA-NeMo/Guardrails  -> LLM safety guardrails (Colang language)
# NVIDIA-NeMo/NeMo-Run    -> Experiment management and launching
# NVIDIA-NeMo/Megatron-Bridge -> NeMo <-> Megatron-LM integration
#
# For your SDET role, focus on: Evaluator (testing), Curator (data validation),
# and Guardrails (safety testing). Skip NeMo core unless doing speech work.
```

Phase 7 Projects

PROJECT 7.1: RUN NeMo EVALUATOR

Objective: Use NVIDIA's evaluation framework to benchmark a model — the tool your team uses for quality gates.

1. Install NeMo Evaluator: `pip install nemo-evaluator` (or use the Docker image).

2. Run a basic evaluation on TinyLlama: HellaSwag (commonsense reasoning) and MMLU subset (knowledge).

3. Study the output format. It provides pass/fail against defined thresholds with full reproducibility metadata (seeds, configs, software versions).

4. Write `tests/test_model_quality.py`:

• Load the evaluation results

• Assert the model scores above a minimum threshold for each benchmark

• This is exactly how model quality gates work in production: if a model fails eval, it does not get deployed.

5. In your Learning Log: What does an MMLU score of 0.45 mean? What would failing a HellaSwag threshold tell you about a model?

✓ Done when: NeMo Evaluator runs locally. You have evaluation scores for TinyLlama. You have a test that asserts on those scores.

PROJECT 7.2: EXPLORE NeMo CURATOR AND GUARDRAILS

Objective: Understand the data and safety tools in the NeMo ecosystem.

Curator:

Clone github.com/NVIDIA-NeMo/Curator. Read the README and tutorials. In Cursor, ask "How does Curator detect and remove PII from training data?" Follow the code path. Write a short summary in your architecture notes.

Guardrails:

Clone github.com/NVIDIA-NeMo/Guardrails. This is the most popular NeMo sub-project (5,900+ stars). It uses a domain-specific language called Colang to define LLM safety rails.

• Run the hello-world example from the Guardrails repo.

• Write a simple guardrail that prevents the model from answering questions about a specific topic.

• In your Learning Log: how could guardrails be tested? What would a "guardrail regression test" look like?

✓ Done when: You have explored both repos. You can explain what each does and why it matters for model quality. You have a running Guardrails example.

PROJECT 7.3: FINAL ARCHITECTURE DOCUMENT

Objective: Produce the document that proves you understand the full stack before day one.

1. Create `docs/architecture_notes.md` (or update if you started in Phase 6).

2. Section 1 — The Full Stack:

```
Draw an ASCII diagram showing the complete lifecycle:
  Training (NeMo/Megatron) -> Data Curation (Curator)
  -> Evaluation (Evaluator) -> Quality Gate (pass/fail)
  -> Optimisation (TRT-LLM) -> Serving (Dynamo)
  -> Safety (Guardrails)    -> User
Label which layer each technology lives in.
```

3. Section 2 — Data Flow: Trace the path of a user prompt from HTTP request to generated token in a full Dynamo deployment. Include: router, prefill worker, NIXL KV transfer, decode worker, token streaming.

4. Section 3 — Where inference-sandbox Fits: Explain how your project tests this stack. Map each test file to the component it validates:

• `test_quantisation.py` → validates precision casting (TRT-LLM quality)

• `test_trtllm_correctness.py` → validates TRT-LLM vs HuggingFace (engine correctness)

- `test_perf_regression.py` → validates throughput (SLO enforcement)
- `test_dynamo_health.py` → validates serving infrastructure (Dynamo health)
- `test_model_quality.py` → validates model quality (NeMo Evaluator)

5. Section 4 — What Would Change at Scale: Describe what is different about testing on a DGX cluster with 8 GPUs per node vs your single 5070 Ti. Mention: tensor parallelism, multi-node KV transfer, network failures, GPU memory management across devices.

6. Section 5 — Open Questions: Write 5 specific, well-formed questions for your first week at NVIDIA. These should demonstrate that you understand the system well enough to ask non-obvious questions.

✓ Done when: Document exists, diagrams are accurate, test mapping is correct, open questions show genuine curiosity and self-awareness.

PROJECT 7.4: FINAL README AND REPO CLEANUP

Objective: Make inference-sandbox presentable for day one.

1. Update README.md with:

- **Overview:** What inference-sandbox is and what it demonstrates
- **Setup:** How to set up the local GPU environment (link to docs/local_gpu_setup.md)
- **Running Benchmarks:** Every benchmark.py flag, with examples
- **Running Tests:** pytest commands for each test category (unit, gpu, integration, slow)
- **Architecture:** Link to docs/architecture_notes.md
- **TRT-LLM:** How to run TRT-LLM benchmarks (link to docs/trtllm_first_run.md)
- **Dynamo:** How to run Dynamo benchmarks (link to docs/dynamo_quickstart.md)
- **Learning Journey:** Honest reflection on the journey from Phase 1 to Phase 7

2. Verify that EVERY command in the README actually works when run on a fresh clone. Broken documentation is worse than no documentation.

3. Ensure CI is green. All GPU tests skip gracefully. All non-GPU tests pass.

✓ Done when: README is complete and every instruction works. CI is green. You would be proud to show this repo to your mentor.

Phase 7 Checkpoint — And Your Final State

■ What You Have Built by May 18:

inference-sandbox is a GitLab repo with:

1. A `benchmark.py` that benchmarks across five backends: Ollama, HuggingFace, vLLM, TRT-LLM, and Dynamo — same interface, swappable engines.

2. A multi-stage Dockerfile and docker-compose.yml with Redis integration.

3. A `Dockerfile.trtllm` extending the NGC TRT-LLM base image.

4. Kubernetes deployment + service + configmap manifests (from Phase 3).

5. A test suite covering: quantisation accuracy, TRT-LLM correctness, performance regression, Dynamo health, and model quality evaluation.

6. Documentation: GPU setup guide, TRT-LLM first run log, Dynamo quickstart, architecture notes, and a professional README.

7. 10 weeks of Learning Log entries showing your progression.

8. Cursor rules showing how you configured AI-assisted development for infrastructure work.

That is not a tutorial completion certificate. That is a portfolio piece. Show it to your mentor on day one.

Updated Repo Structure

Phases 4–7 additions shown in bold.

```
inference-sandbox/
|-- benchmark.py                  # Core benchmarking (5 backends by Phase 7)
|-- k8s_test.py                   # Kubernetes Python client test (Phase 3)
|-- requirements.txt              # Python dependencies
|-- Dockerfile                    # Multi-stage build (Phase 2)
|-- Dockerfile.trtllm             # * TRT-LLM NGC base extension (Phase 5)
|-- docker-compose.yml            # App + Redis stack (Phase 2)
|-- .gitlab-ci.yml                # CI pipeline
|-- .gitignore
|-- .cursor/
|   `-- rules/
|       |-- nvidia-stack.mdc      # * NVIDIA stack context (Phase 4)
|       `-- sdet-mindset.mdc      # * Testing-first guidance (Phase 4)
|-- README.md                     # Professional guide (Phase 7)
|-- LEARNING_LOG.md               # Weekly reflections (all phases)
|-- golden_baseline.json          # * TRT-LLM perf baseline (Phase 5)
|-- k8s/
|   |-- deployment.yaml
|   |-- service.yaml
|   `-- configmap.yaml
|-- benchmarks/
|   |-- ollama_bench.py           # * Ollama benchmarks (Phase 4)
|   |-- hf_bench.py              # * HuggingFace benchmarks (Phase 4)
|   |-- vllm_bench.py            # * vLLM concurrent bench (Phase 4)
|   |-- vram_observer.py         # * VRAM pressure tool (Phase 4)
|   `-- trtllm_basic.py          # * TRT-LLM first inference (Phase 5)
|-- tests/
|   |-- test_quantisation.py      # * FP32/FP16/BF16 perplexity (Phase 4)
|   |-- test_trtllm_correctness.py # * TRT-LLM vs HuggingFace (Phase 5)
|   |-- test_perf_regression.py   # * Throughput regression gate (Phase 5)
|   |-- test_bench_sanity.py      # * Custom vs trtllm-bench (Phase 5)
|   |-- test_dynamo_health.py     # * Dynamo endpoint health (Phase 6)
|   `-- test_model_quality.py     # * NeMo Evaluator quality (Phase 7)
`-- docs/
    |-- local_gpu_setup.md        # * GPU env setup log (Phase 4 setup)
    |-- vram_vs_seqlen.png        # * VRAM plot (Phase 4)
    |-- vram_vs_batch.png         # * VRAM plot (Phase 4)
    |-- trtllm_first_run.md       # * TRT-LLM exact commands (Phase 5)
    |-- dynamo_quickstart.md      # * Dynamo setup log (Phase 6)
    `-- architecture_notes.md     # * Full stack diagram (Phase 7)
```

Glossary

Terms you will hear week one. Combined from both the original and revised guides.

AutoDeploy

TRT-LLM feature (beta) that automatically optimises HuggingFace models for TRT-LLM inference without manual conversion or engine building. The “easy path” for supported models.

Autoregressive Decoding

The process of generating text one token at a time, where each token depends on all previous tokens. The core loop of LLM inference.

BF16

Brain Float 16. 16-bit floating point format with the same exponent range as FP32, making it numerically stable for training. Preferred over FP16 on Ampere+ GPUs.

Continuous Batching

A serving strategy where new requests join a running batch as soon as a slot opens, without waiting for all current requests to finish. Massively improves throughput.

cuDNN

NVIDIA’s deep learning primitives library. Provides optimised CUDA implementations of convolution, attention, and other DL operations. Bundled in CUDA runtime images.

Device Plugin

A Kubernetes component (nvidia/k8s-device-plugin) that exposes GPU resources to the K8s scheduler, enabling pods to request GPUs via nvidia.com/gpu: 1.

Disaggregated Prefill/Decode

Dynamo’s core architectural innovation. Routes the compute-bound prefill phase to GPUs optimised for FLOPS and the memory-bandwidth-bound decode phase to GPUs optimised for bandwidth. Scales each independently.

Dynamo Planner

Dynamo’s dynamic GPU scheduler. Monitors real-time request patterns and SLO targets, then shifts GPUs between prefill and decode pools based on demand. Think of it as Kubernetes HPA but for inference pipeline stages.

EOS Token

End of Sequence. A special token the model generates to signal it has finished its response.

FP8

NVIDIA H100-native 8-bit float format. Two variants: E4M3 (better range) and E5M2 (better precision). TRT-LLM’s default quantisation on H100, ~2x faster than FP16.

Greedy Decoding

Always selecting the highest-probability next token. Deterministic — same prompt always produces same output. Used as the reference for correctness tests.

HBM3

High Bandwidth Memory 3. The VRAM type on H100 GPUs. 80 GB capacity, 3.35 TB/s bandwidth. The decode phase is bottlenecked by this bandwidth.

KV Cache

Key-Value cache storing attention tensors from previous tokens during autoregressive decode. Avoids recomputing attention over the full sequence at each step. VRAM-intensive.

LLM API

TRT-LLM v1.0+ Python API (from tensorrt_llm import LLM). Loads models directly from HuggingFace with automatic optimisation. Replaces the legacy checkpoint-conversion → build → deploy pipeline for most use cases.

LoRA

Low-Rank Adaptation. A fine-tuning technique that adds small trainable adapter matrices to frozen model weights. Only trains ~0.1% of parameters.

Megatron-LM

NVIDIA's framework for training trillion-parameter models using tensor, pipeline, and sequence parallelism across thousands of GPUs. NeMo wraps it.

NeMo Curator

GPU-accelerated data curation toolkit for training data. Handles deduplication, PII removal, language filtering, and quality scoring. Data quality directly determines model quality.

NeMo Evaluator

Model evaluation framework. Runs 100+ standardised benchmarks (MMLU, GSM8K, HumanEval, etc.) with reproducibility guarantees. Used as quality gates before model deployment.

NeMo Guardrails

Toolkit for adding programmable safety guardrails to LLM applications using the Colang domain-specific language. Covers input moderation, fact-checking, topic control, and jailbreak prevention.

NGC

NVIDIA GPU Cloud. NVIDIA's registry of pre-built containers, pre-trained models, and SDKs. nvcr.io is the container registry URL.

NIXL (NVIDIA Inference Xfer Library)

Dynamo's KV cache transfer protocol. Moves KV cache between prefill and decode GPUs via NVLink or RDMA with microsecond latency. The "glue" that makes disaggregation work.

NVLink

NVIDIA's proprietary inter-GPU interconnect. On H100 NVL systems: 900 GB/s bidirectional bandwidth between GPUs. Used by Dynamo's NIXL for KV cache transfers.

ONNX

Open Neural Network Exchange. Intermediate format for exporting models from PyTorch/TF for use with TensorRT or other inference runtimes.

p99 Latency

The latency at which 99% of requests complete. Used in production SLOs because it captures tail latency (the experience of the worst-off 1% of users). Average latency hides bad tails.

Perplexity

Measure of language model prediction quality. Exponential of average negative log-likelihood. Lower = better. Used to compare model quality across quantisation levels.

Pipeline Parallelism

Splitting model layers across multiple GPUs in sequence (GPU 0 runs layers 1–16, GPU 1 runs layers 17–32). Used for very deep models.

Prefill

The phase of LLM inference where the full input prompt is processed in one forward pass and the KV cache is populated. Determines TTFT.

SLO

Service Level Objective. A target value for a metric (e.g., p99 TTFT < 200ms). Your test suite enforces SLOs — that is the SDET's job.

Smart Router

Dynamo component that uses radix-tree-based KV cache tracking to route requests to GPUs that already have relevant cached context, reducing redundant prefill computation.

Speculative Decoding

Use a small draft model to propose K tokens, then verify them in one pass of the large model. Reduces TPOT at the cost of running an extra small model.

Tensor Parallelism

Splitting a matrix multiplication across multiple GPUs (each computes a shard). Requires an all-reduce communication step after each layer.

TPOT

Time Per Output Token. Average latency between consecutive generated tokens during the decode phase. Bottlenecked by memory bandwidth.

TTFT

Time To First Token. Latency from request arrival to when the first output token is generated. Bottlenecked by the prefill compute phase.

trtllm-bench

TRT-LLM's built-in benchmarking tool. Reports TTFT, TPOT, and throughput at p50/p90/p99 percentiles. The reference implementation for inference performance measurement.

trtllm-serve

TRT-LLM's built-in OpenAI-compatible API server. Single command to serve any supported model. Replaces manual Triton Inference Server setup for most deployment scenarios.

10 weeks from now,

you will have built something real.

Not read about it. Not watched a tutorial about it.

Built it, broken it, debugged it, and shipped it.

That is the difference between a resume line

and something you actually know.

See you on May 18.

— NVIDIA Deep Learning Infrastructure